

PM: Megan/Max Harvey – ST10438637

Unathi Mudzengi – ST10453134

Nuha Grimwood – ST10452224

Luis De Souza – ST10307204

INSY7314

Part 1 Documentation

07/09/2026

Table of Contents

System Overview	2
YouTube Link	2
Architecture Diagram	3
.....	4
Security Design.....	6
Password Hashing	6
JWT Token-Based Authentication	6
Input Validation and Normalisation	7
HTTPS and TLS	7
Secure Error Handling.....	7
Environment Variables and Secret Management	8
Protected Routes.....	8
Security Headers with Helmet	8
CORS Restricted	9
Request Payload Limits.....	9
Reference List.....	9
Postman Testing.....	10
HustleHub API Collection Overview.....	10
Individual Requests.....	11

System Overview

The HustleHub+ system is a platform where freelancers who offer services have access to a marketplace where clients can book the freelance service. It is a centralised platform that allows freelancers to advertise and manage their gigs, while clients can browse the services and book a service that they might need. Additionally, the system implements financial tracking as an additional feature for the freelancers use. When a booking is created the system will reflect that income through a simulated transaction, while also calculating the tax of that incoming amount. The system provides a financial dashboard for freelancers to view their earnings and income taxes at a glance.

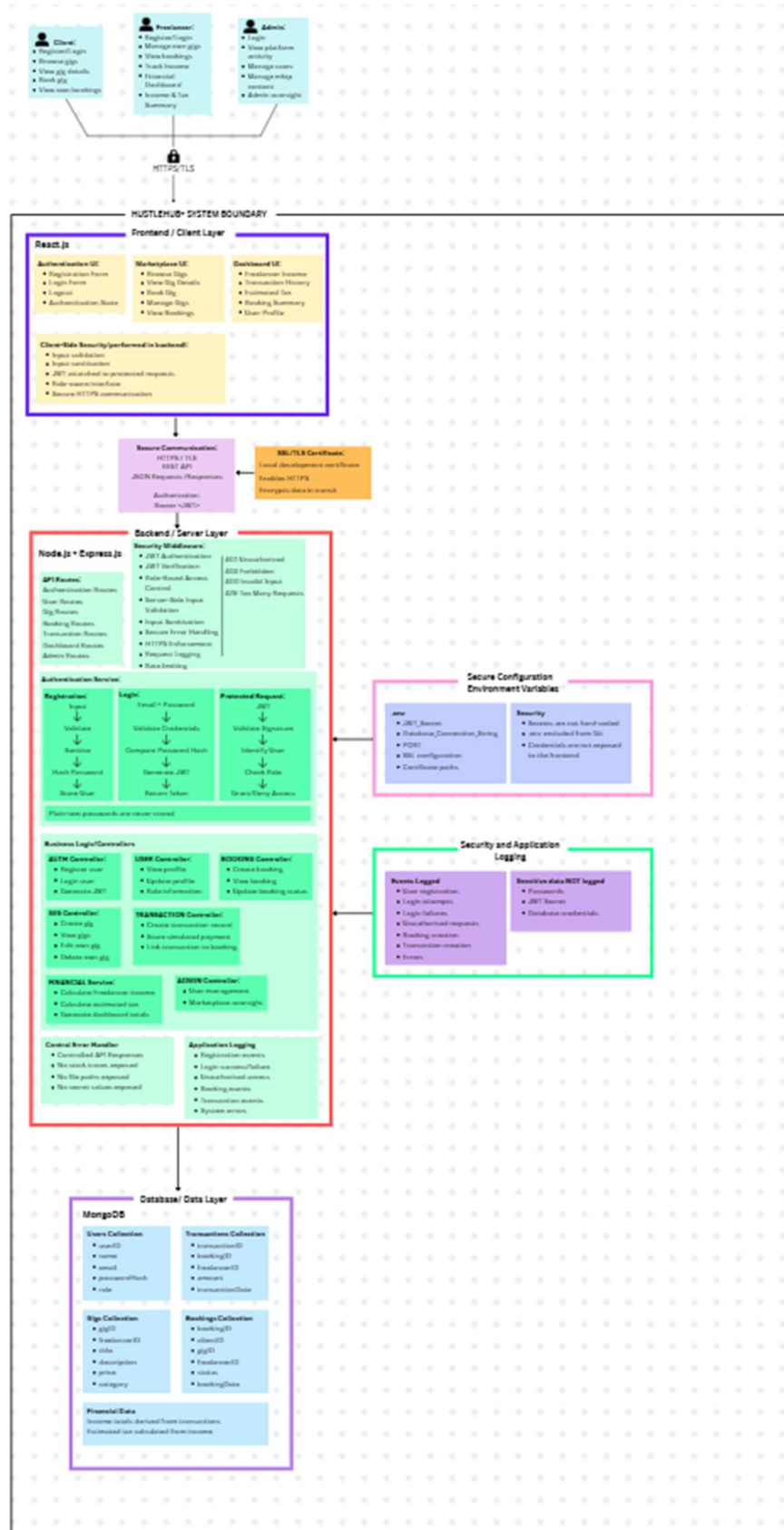
The HustleHub+ app follows a MERN architecture using MongoDB, Express.js, React.js and Node.js. React is in charge of the frontend or client-side operations, whereas Node and Express manage the backend REST API actions as well as the application logic. MongoDB provides consistent storage for app data such as users, gigs, bookings and transactions.

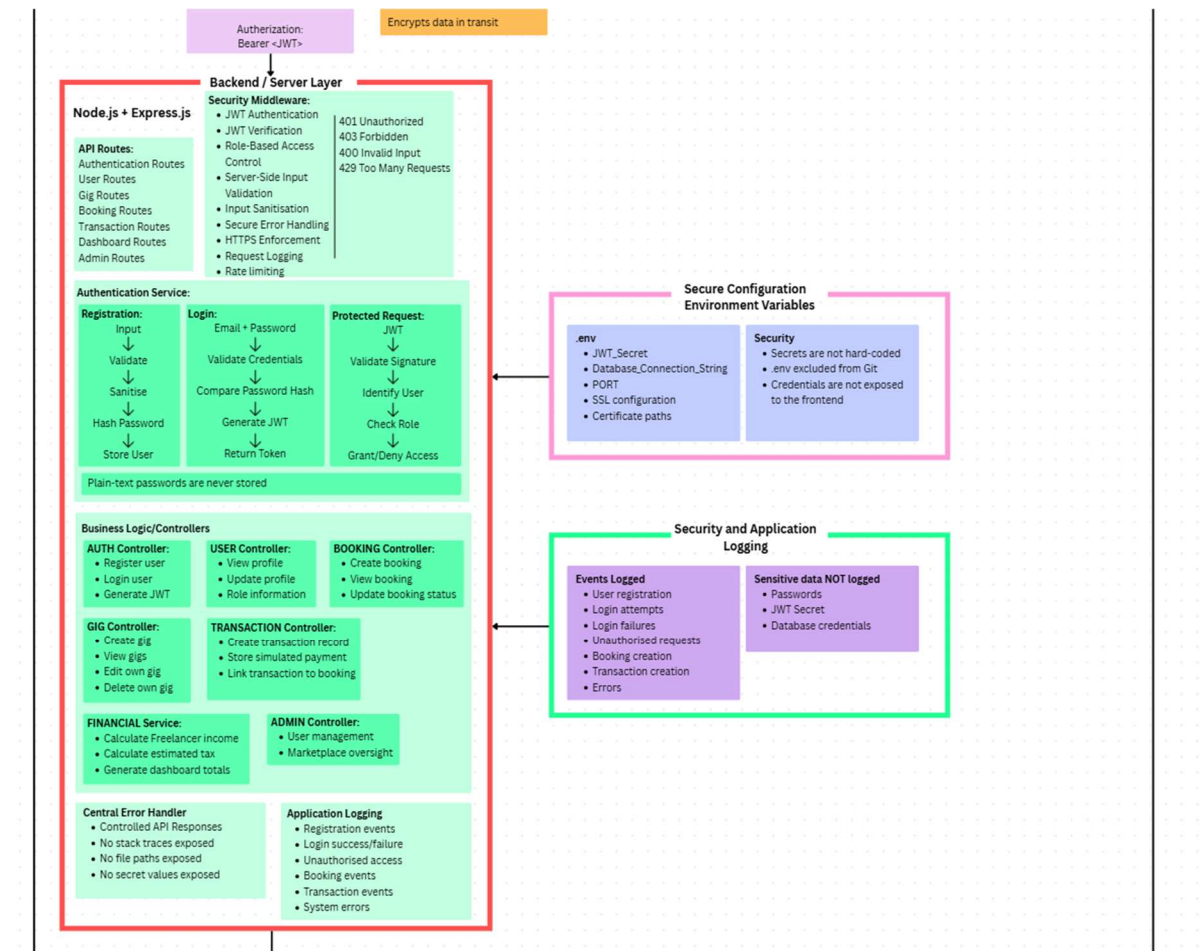
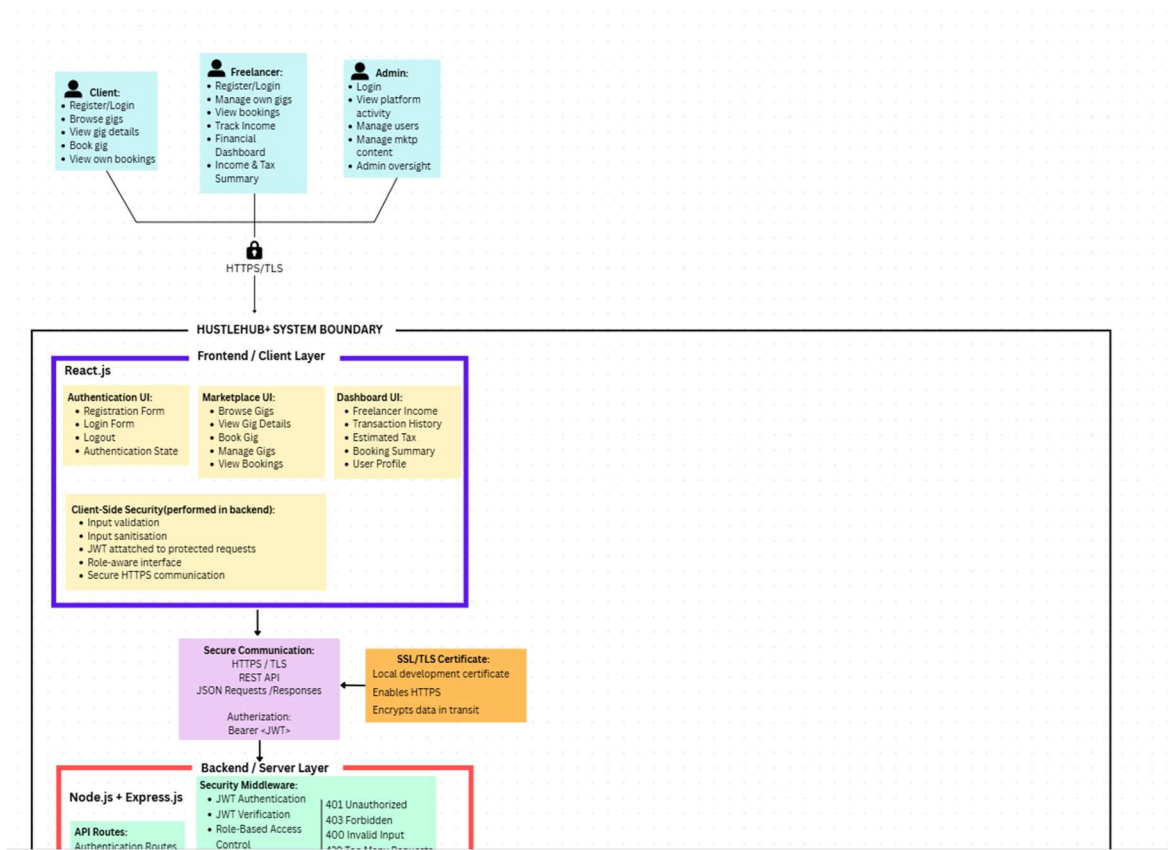
Part 1 of this project focuses on building a secure foundation for the backend of the system. The backend foundation was built using Node.js and Express.js and can currently perform actions such as user registration, login and access to a protected user profile endpoint. Currently the user data is saved in an in-memory data structure. The MongoDB implementation as well as a larger range of functionality will be incorporated into the system in the following project parts.

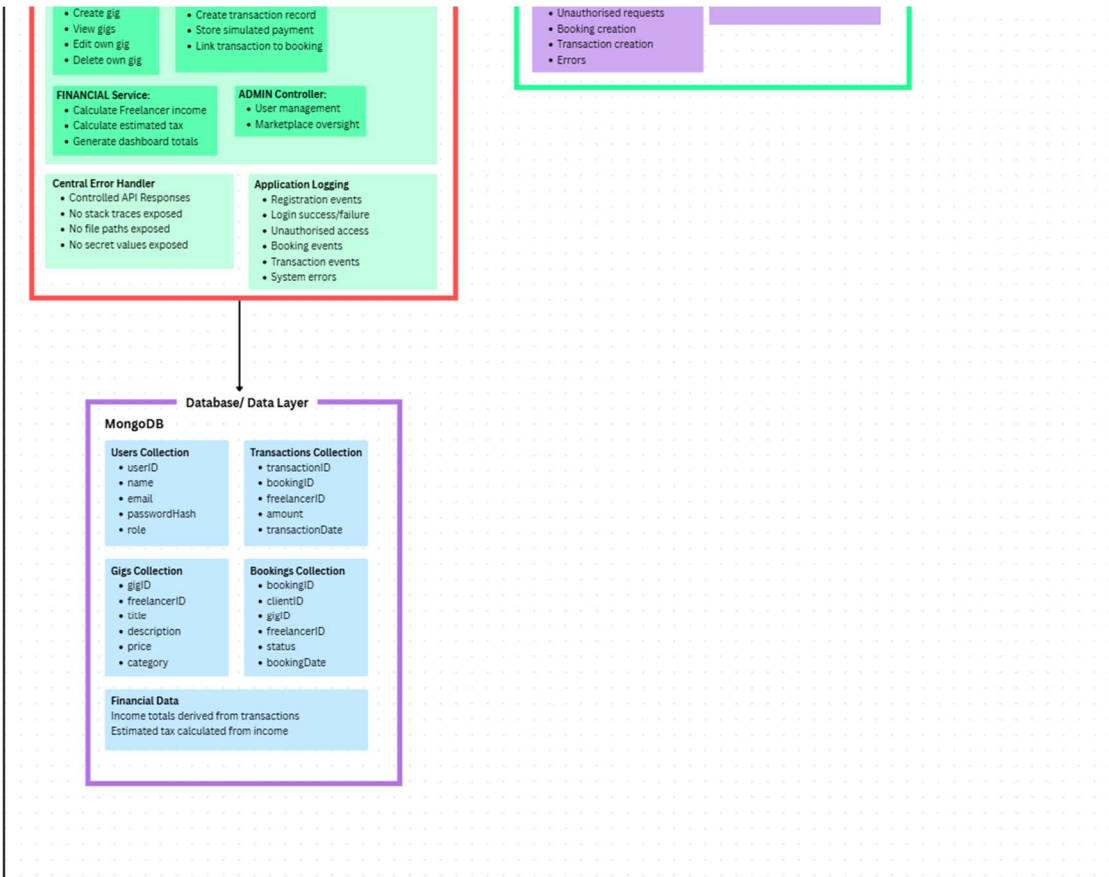
YouTube Link

<https://youtu.be/W0NX8Btqt5M>

Architecture Diagram







Security Design

Security was a focal point when addressing the architecture of the HustleHub+ system because the platform will be processing sensitive information, including user credentials, authentication information, transaction records, income information and estimated tax values. Due to the nature of the functionality of the system, there are several security implications in place from the initial system creation. The security features that were put into place to protect the system consist of password hashing, JWT authentication, input validation, HTTPS, secure error handling, environment-based configuration, security headers, restricted CORS and request-size limits.

Password Hashing

Password hashing converts a user's password into a one-way hash rather than storing the password as plain text (Manico & Detlefsen, 2015). In HustleHub+, the developers used 'bcrypt' to hash passwords during the registration process with a work factor of 12. The only version of the password that is saved in the database is the hashed version. During the login process. The action of 'bcrypt.compare()' checks the submitted password against the hashed password that is stored. The password and hashed password are not included in the API responses. The reason this security feature was used is because passwords should never be stored as plain text (Manico & Detlefsen, 2015). By hashing the passwords, the risk of exposing users' original passwords is greatly reduced.

JWT Token-Based Authentication

JSON Web Tokens (JWTs) is a security measure where a token-based method of identifying authenticated users is utilised when they are accessing protected API endpoints (Manico & Detlefsen, 2015). After a successful login, HustleHub+ generates a signed JWT containing the userID and role. The token is only valid for an hour by default. When accessing protected API routes, the 'protect' middleware retrieves the Bearer token from the 'Authorization' header and then gets verified through 'jwt.verify()'. The valid token information gets attached to the 'req.user', if this is missing, invalid or expired tokens will result in a 401 response. The developers used this protection because it allows for the backend to protect API endpoints as well as verifying that the request is coming from an authenticated user before allowing access (Manico & Detlefsen, 2015). By including user roles in Part 1, there is a foundation for role-based access control in the upcoming parts.

Input Validation and Normalisation

Input validation checks is the process of checking the incoming user data to see if it meets the expected requirements before it is processed by the application (Manico & Detlefsen, 2015). Normalisation creates a consistent representation of accepted input. HustleHub+ uses a dedicated middleware layer to validate the registration and login inputs, where it checks for required fields, verifies the values are strings, validates the email format, and restricts registration passwords between 8 and 72 characters long. Names are trimmed (of whitespace), while email addresses are trimmed and converted to lowercase before reaching the app controller. In the context of HustleHub+, the API cannot automatically be trusted. By using validation, the system can prevent incorrectly structured or unacceptable data from being processed and makes the attack surface smaller against uncontrolled user input.

HTTPS and TLS

HTTPS uses Transport Layer Security (TLS) to encrypt communication between a client and the server (Manico & Detlefsen, 2015). HustleHub+ can run the Express application through Node.js's 'https' module when 'USE_HTTPS=true'. The server loads a local SSL private key and certificate from paths configured through environment variables (.env). The developers of HustleHub+ used a self-signed certificate for the Part 1 local development. Sensitive information such as login credentials and JWTs travels between the client and backend. To reduce the risk of being intercepted in a readable form, HTTPS implements encryption of the information while it is in transit. For production level deployment, a trusted certificate would be required rather than a development level self-signed certificate.

Secure Error Handling

Secure error handling is in charge of controlling the information that is returned when the application encounters an error (Manico & Detlefsen, 2015). HustleHub+ makes use of a centralised Express error-handling middleware. When unexpected errors occur, they are logged in the server, whereas the client receives a generic '500' response rather than an error message that exposes internal application details. Authentication also makes use of the same error message for an unknown email address or an incorrect password. Detailed errors can expose information about the internal application to a malicious user. Controlled responses will reduce the chances of sensitive information being disclosed while still allowing the server side to receive and record errors for debugging.

Environment Variables and Secret Management

Environment variables separate configuration and sensitive values from the applications source code (Manico & Detlefsen, 2015). HustleHub+ loads configuration using 'dotenv'. The '.env' file contains values such as 'JWT_SECRET', JWT expiry, SSL certificate paths, frontend origin, port and the HTTPS configuration. The application will not generate authentication tokens if JWT_SECRET has not been configured. In the context of HustleHub+, sensitive values such as the JWT signing secret should not be hard coded into the application source files. By separating secrets and configuration from the source code, it is easier to protect and allows for different values to be used in development and production environments.

Protected Routes

Protected routes are API endpoints that require successful authentication before they can be accessed (Manico & Detlefsen, 2015). While routes such as registration or login are public, routes that are more specific and hold private information such as '/api/auth/me' need to be protected using the 'protect' middleware. The middleware requires the Bearer JWT, verifies it and then places the authenticates user's ID and role in 're.user'. In the case of a request without a valid token, the request will be rejected. HustleHub+ deals with user-specific information which should not be available for unauthenticated users. By protecting endpoints, it is ensured that authentication is checked before restricted resources are accessed and provides the foundation for more detailed Client, Freelancer and Admin permissions later.

Security Headers with Helmet

HTTP security headers provide browsers with instructions that can restrict potentially unsafe behaviour (Manico & Detlefsen, 2015). HustleHub+ uses 'helmet' to configure security-related response headers. The application defines a Content Security Policy (CSP), disables object sources, prevents framing through 'frameAncestors' as well as disables Express's X-Powered-By header. Unnecessary information disclosure is reduced by putting these measures in place, while also providing additional browser-side protection against threats such as content injection and clickjacking as the frontend continues to be developed.

CORS Restricted

Cross-Origin Resource Sharing (CORS) determines which browser origins are permitted to make cross-origin requests to the API. HustleHub+ configures CORS using the 'CLIENT_ORIGIN' environment variable, instead of accepting requests from any and every browser origin. CORS also specifies what HTTP methods and request headers are permitted. Restricting CORS prevents arbitrary websites from being granted browser-based cross-origin access to the HustleHub+ API. It also gives the team explicit control over the frontend origin that is expected to communicate with the backend.

Request Payload Limits

A request payload limit controls the maximum size of JSON data that the Express server will accept (Manico & Detlefsen, 2015). HustleHub+ limits JSON request bodies to 10kb. The authentication API only requires small JSON requests. Restricting unnecessarily large request bodies reduces resource consumption and provides an additional defensive measure against oversized requests.

Reference List

Manico, Jim., & Detlefsen, August. (2015). *Iron-clad Java : building secure web applications*. Mc Graw Hill Education.


```
POST 11. Login - Non-existent Email
https://localhost4000/api/auth/login
401 Unauthorized 5 ms 1.004 KB
PASS Status is 401
PASS Same generic message (no user enumeration)

POST 12. Login - Missing Fields
https://localhost4000/api/auth/login
400 Bad Request 6 ms 1.009 KB
PASS Status is 400

GET 13. Me - No Token
https://localhost4000/api/auth/me
401 Unauthorized 5 ms 1.008 KB
PASS Status is 401

GET 14. Me - Malformed Header
https://localhost4000/api/auth/me
401 Unauthorized 10 ms 1.008 KB
PASS Status is 401
```

```
GET 15. Me - Invalid/Tampered Token
https://localhost4000/api/auth/me
401 Unauthorized 5 ms 1.003 KB
PASS Status is 401
PASS Generic message

GET 16. Me - Valid Token
https://localhost4000/api/auth/me
200 OK 5 ms 1.062 KB
PASS Status is 200
PASS Profile matches test user

GET 17. 404 - Unknown Route
https://localhost4000/api/auth/nope
404 Not Found 5 ms 991 B
PASS Status is 404

POST 18. Payload Too Large
https://localhost4000/api/auth/register
500 Internal Server Error 6 ms 1.033 KB
PASS Oversized payload rejected
```

Individual Requests

```
HustleHub API / Health / 1. GET /health
GET [baseUrl]/health
Send

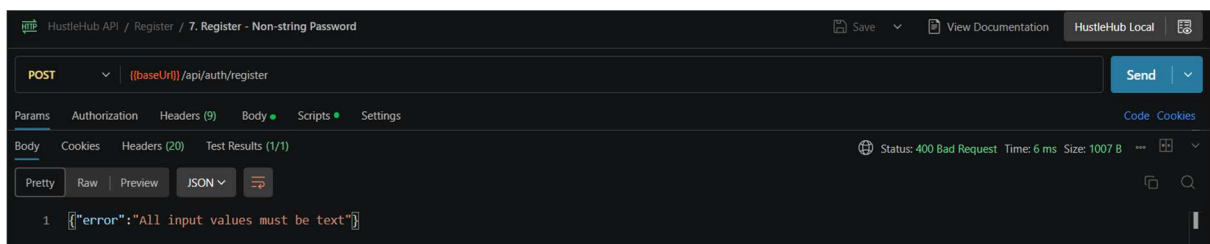
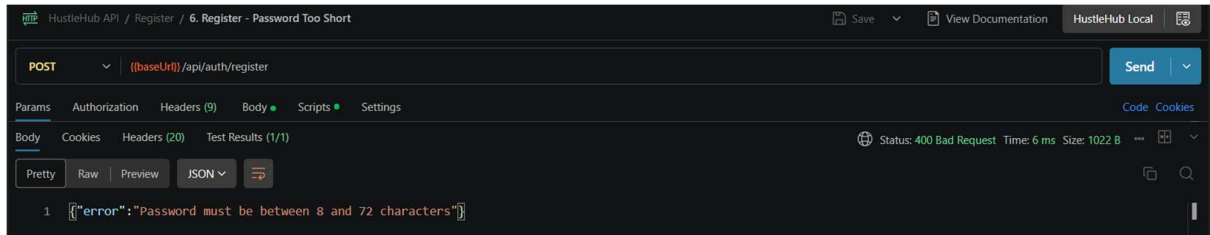
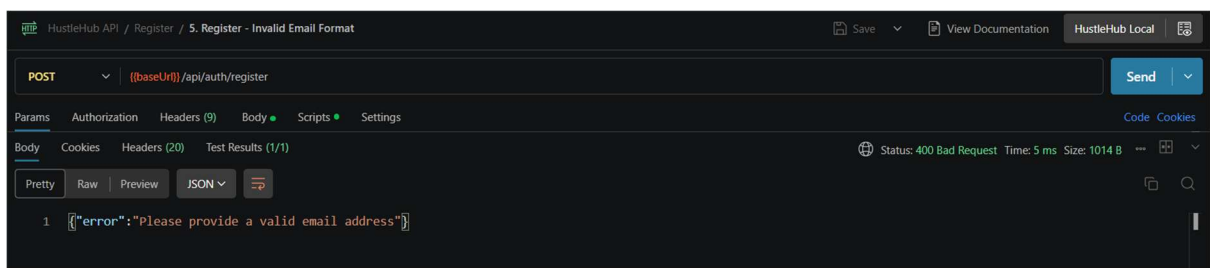
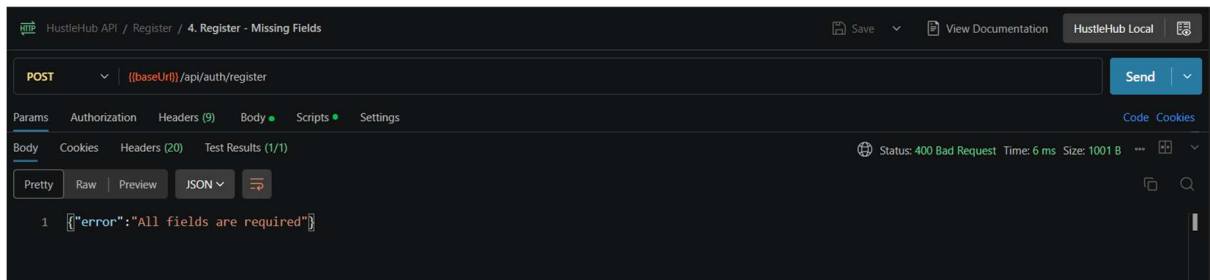
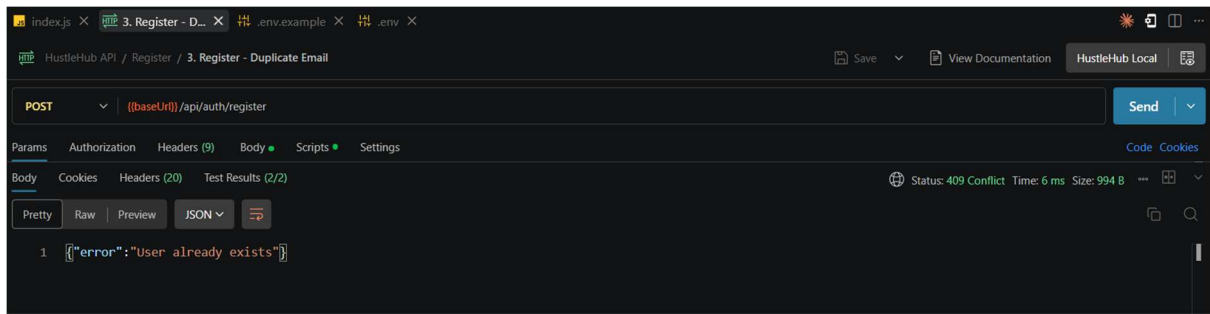
Params Authorization Headers (7) Body Scripts Settings
Body Cookies Headers (20) Test Results (2/2)
Status: 200 OK Time: 9 ms Size: 991 B
Pretty Raw Preview JSON
```

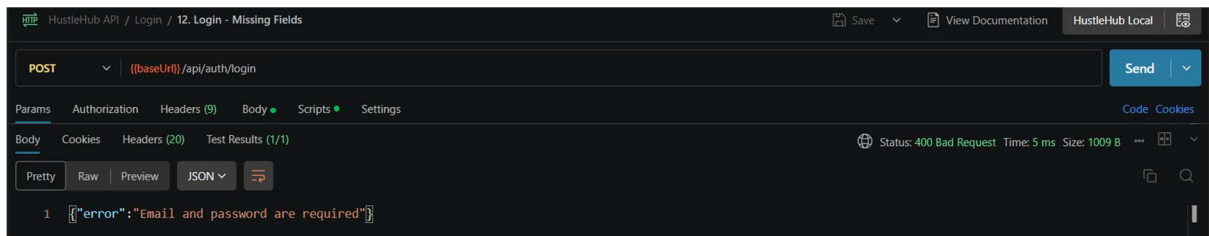
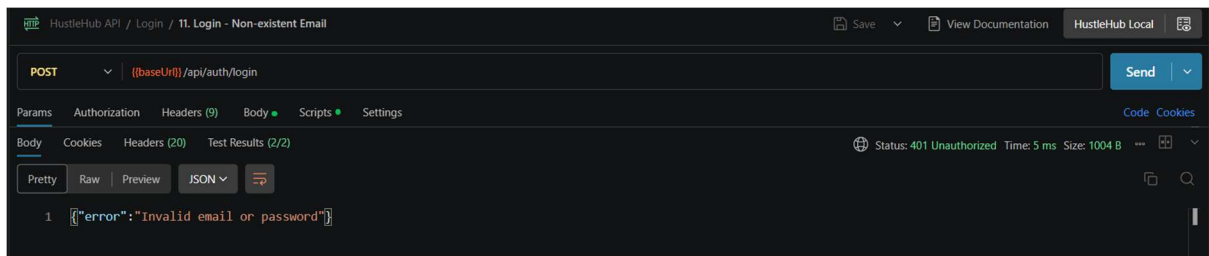
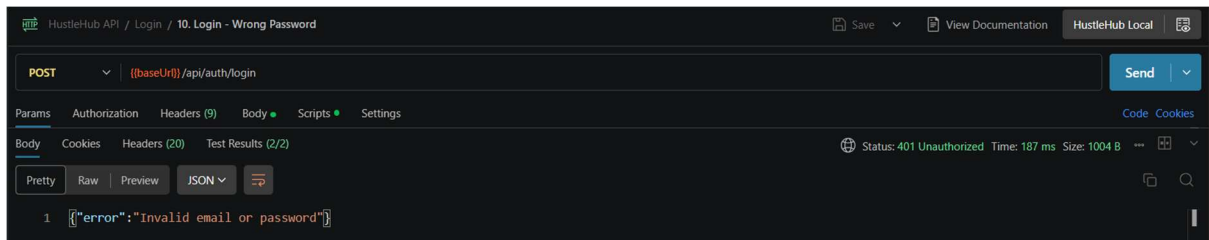
```
1 [{"status": "OK", "protocol": "HTTPS"}]
```

```
Index.js x 2. Register - H... x .env.example x .env x
HustleHub API / Register / 2. Register - Happy Path
POST [baseUrl]/api/auth/register
Send

Params Authorization Headers (9) Body Scripts Settings
Body Cookies Headers (20) Test Results (2/2)
Status: 201 Created Time: 196 ms Size: 1.08 KB
Pretty Raw Preview JSON
```

```
1 [{"message": "User registered successfully", "data": {"id": "u1", "fullName": "Person Three Tester", "email": "insy.test@hustlehub.test", "role": "user"}}]
```





HustleHub API / Protected / 13. Me - No Token

GET `{{baseUri}}/api/auth/me` Send

Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (20) Test Results (1/1) Status: 401 Unauthorized Time: 5 ms Size: 1008 B

Pretty Raw Preview JSON

```
1 [{"error": "Authentication token required"}]
```

HustleHub API / Protected / 14. Me - Malformed Header

GET `{{baseUri}}/api/auth/me` Send

Params Authorization Headers (8) Body Scripts Settings

Body Cookies Headers (20) Test Results (1/1) Status: 401 Unauthorized Time: 5 ms Size: 1008 B

Pretty Raw Preview JSON

```
1 [{"error": "Authentication token required"}]
```

HustleHub API / Security + Negative Tests / 17. 404 - Unknown Route

GET `{{baseUri}}/api/auth/nope` Send

Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (20) Test Results (1/1) Status: 404 Not Found Time: 5 ms Size: 991 B

Pretty Raw Preview JSON

```
1 [{"error": "Route not found"}]
```

HustleHub API / Security + Negative Tests / 18. Payload Too Large

POST `{{baseUri}}/api/auth/register` Send

Params Authorization Headers (9) Body Scripts Settings

Body Cookies Headers (20) Test Results (1/1) Status: 500 Internal Server Error Time: 6 ms Size: 1.01 KB

Pretty Raw Preview JSON

```
1 [{"error": "Something went wrong. Please try again later."}]
```